MACHINE LEARNING

REPORT: Multiclass Classification

VehicleVision: Robust Vehicle Classification

GitHub: [https://github.com/Mayank-parmar1/ML-Project1]

1. Introduction

The goal of this project is to build a machine learning system that predicts a vehicle type from its technical specifications and performance measurements. The task is a three-class classification problem. Each vehicle is classified as a Sedan, SUV, or Sports Car.

The dataset contains numerical measurements such as engine displacement, horsepower, weight, dimensions, fuel efficiency, acceleration, braking distance, tire width, drag coefficient, and battery capacity. The competition also notes that evaluation measurements may contain additional noise.

The project workflow included:

- Exploratory data analysis and data-quality checks
- Duplicate-profile handling and preprocessing
- Training and comparison of several individual classification models
- Probability blending and controlled parameter experiments
- Generation and validation of Kaggle submission files

The best submitted configuration achieved a Kaggle leaderboard score of 0.90600 (90.6%). This report explains the final model in simple terms and separates that leaderboard score from local validation results.

2. Dataset Overview

The competition provided three CSV files:

File	Purpose	Rows
vehicle_train_10000.csv	Training measurements and target class	10,000
vehicle_test.csv	Measurements requiring predictions	2,500
vehicle_sample_submission.csv	Required submission format and ID order	2,500

2.1 Target Variable

The target column is vehicle_class. It has three valid labels:

Label	Vehicle type	Share in original training data (approx.)
0	Sedan	33.89%
1	SUV	33.26%
2	Sports Car	32.85%

The three classes are reasonably balanced. Therefore, accuracy is a useful primary metric, while macro F1 was also monitored during local validation.

2.2 Input Features

There are 20 numerical vehicle attributes. They include:

- Engine displacement, horsepower, torque, and number of engine cylinders
- Vehicle weight, wheelbase, length, width, height, and ground clearance
- Fuel tank capacity, city fuel efficiency, highway fuel efficiency, and battery capacity
- Turning radius, cargo capacity, 0-100 km/h acceleration time, and braking distance
- Tire width and drag coefficient

2.3 Data Quality Observations

- The original training file contained 10,000 rows and 20 measurement features.
- There were 7,500 unique feature profiles. The remaining 2,500 rows were duplicate measurement profiles with consistent labels.
- No missing values were found during the initial inspection. Median imputation was kept as a safeguard in the pipeline.
- The vehicle_id column identifies a record and was not used as a predictive feature.

3. Exploratory Data Analysis (EDA)

EDA was used to understand the target distribution, feature ranges, duplicate records, and possible data-quality issues. Main findings were:

- All 20 model input columns were numerical measurements.
- The target classes were close to one-third each, so there was no strong class imbalance.
- Exact duplicate profiles were common enough to require explicit handling.
- Potential outliers were inspected with the IQR rule. They were not automatically removed because extreme vehicle measurements may still contain useful information.
- The competition description warns that hidden evaluation measurements can contain additional random measurement error.

4. Preprocessing Pipeline

4.1 Duplicate Profile Handling

Before splitting the data for local validation, identical rows were identified using the 20 measurement features, excluding vehicle_id and vehicle_class. The labels of identical profiles were checked. Because matching profiles had matching labels, one copy of each profile was retained for the duplicate-safe experiments.

4.2 Missing Values

The initial files did not contain missing feature values. Median imputation was retained as a defensive step so the code can handle a missing numeric value if one appears later. The medians are calculated from training data, not from the test labels.

4.3 ID Exclusion

vehicle_id was excluded from the model features. It is a record identifier, not a physical vehicle property. Using it as a normal numeric input could teach the model accidental patterns that do not generalize.

4.4 Scaling

StandardScaler was applied in the Logistic Regression pipeline. Scaling centers features and puts them on comparable numeric scales. CatBoost uses decision trees and does not require the same standardization step.

5. Feature Engineering and Noise Augmentation

5.1 Polynomial Features for Logistic Regression

The Logistic Regression model used degree-2 PolynomialFeatures. Along with the original measurements, this creates squared terms and pairwise interactions. For example, the model can learn that horsepower may have a different relationship with vehicle class at different vehicle weights. The features were then standardized.

5.2 Training Noise Augmentation for CatBoost

To make CatBoost less sensitive to small measurement changes, one extra perturbed copy of each unique training row was created. Gaussian noise was added with a standard deviation equal to 2.5% of that feature's training standard deviation. The number of engine cylinders was kept unchanged because it is a discrete measurement.

This augmentation was applied to CatBoost training data only. The purpose was to expose the model to small variations, not to create new target labels or use hidden test labels.

6. Train-Validation Strategy

For a fair local diagnostic, exact duplicate measurement profiles were consolidated before the split. A stratified 80/20 split with random_state=42 was used on the unique profiles:

Partition	Unique profiles	Purpose
Training split	6,000	Fit the models
Local holdout	1,500	Evaluate on profiles not used for fitting

Cross-validation was also used when comparing experiment configurations. The public Kaggle score is reported separately because it is measured on the competition evaluation data, which is not the same as the local holdout.

7. Supervised Models Experimented With

The required individual models and several additional alternatives were tried. This helped establish a simple baseline before selecting the final probability blend.

Model	Best result in the initial local holdout sweep	Observation
Logistic Regression	88.93%	Strong baseline; selected as the main component
Polynomial SVM	88.47%	Competitive, but not the final choice
RBF SVM	88.13%	Did not improve the final Kaggle result when blended
CatBoost	87.73%	Captures nonlinear relationships
XGBoost	87.60%	Tested alone and as a small blend component
Gradient Boosting	87.47%	Competitive tree-based alternative
LightGBM	87.33%	Tested as an individual model
Random Forest	86.73%	Lower local holdout accuracy in the initial sweep
Linear Regression	73.27%	Converted numeric outputs to class labels; weaker fit for this task

These figures are from the initial local holdout model comparison. They are not Kaggle leaderboard scores and should not be compared directly with the final 0.90600 competition score.

8. Final Model: Weighted Probability Blend

The best submission was produced by combining two independently trained models: Logistic Regression and CatBoost. This is a probability blend, not a stacking model.

8.1 Logistic Regression Settings

Setting	Value
Polynomial features	Degree 2; include_bias=False
Scaling	StandardScaler
Regularization	L2
C	1.0
Solver	lbfgs
Maximum iterations	5,000
Random state	42

8.2 CatBoost Settings

Setting	Value
Iterations	650
Tree depth	7
Learning rate	0.035
L2 leaf regularization	5.0
Random strength	1.0
Random seed	42
Training augmentation	One extra row per profile; 2.5% Gaussian noise

8.3 How the Blend Works

Each model outputs a probability for classes 0, 1, and 2. The probabilities are combined class by class using these weights:

> **Final probability =** 0.70 × Logistic Regression probability + 0.30 × CatBoost probability

The predicted class is the class with the highest blended probability. In simple terms, Logistic Regression contributes 70% of the decision and CatBoost contributes 30%.

9. Final Evaluation and Results

9.1 Kaggle Submission Results

Submission / experiment	Kaggle score	Decision
Logistic Regression baseline	0.89800	Starting benchmark
70:30 Logistic + CatBoost initial blend	0.90300	Improved over baseline
70:30 blend, CatBoost depth 6	0.90400	Previous best
70:30 blend, CatBoost depth 7 (final)	0.90600	Best submitted result

Final Kaggle score: 0.90600, equivalent to 90.6% accuracy.

9.2 Local Holdout Results

Model	Training accuracy	Holdout accuracy	Holdout macro F1
Logistic Regression	90.12%	88.93%	88.96%
CatBoost depth 7	94.68%	87.40%	87.46%
Final 70:30 blend	91.43%	88.60%	88.63%

The local holdout score and Kaggle score are different measurements on different evaluation subsets. The final Kaggle score is the reported competition result; the local holdout provides a separate check on generalization.

10. Kaggle Submission

The final submission file was generated from the 70:30 probability blend using the full available training set after duplicate-profile handling. The model predicted one class for each of the 2,500 test rows.

- Final file: vehicle_submission_lr70_cat30_depth7_noise025.csv
- Required columns: vehicle_id, vehicle_class
- The vehicle_id values and row order were checked against the sample submission.
- Predicted labels are restricted to 0, 1, and 2.

11. Overfitting and Noise Robustness Check

The local training-versus-holdout comparison shows a generalization gap. The gap is largest for CatBoost alone and smaller for the final blend, which is one reason the blend was preferred over using CatBoost by itself.

Model	Train accuracy	Holdout accuracy	Gap
Logistic Regression	90.12%	88.93%	1.19 percentage points
CatBoost depth 7	94.68%	87.40%	7.28 percentage points
70:30 blend	91.43%	88.60%	2.83 percentage points

A noise stress test added small random perturbations to holdout measurements. The blend's mean accuracy was around 88.5% at 1% added noise, 88.4% at 2.5%, and 88.5% at 5%. These tests are useful diagnostics, but the simulated noise may not perfectly match the hidden competition measurement process.

Conclusion of the check: there is some overfitting risk, especially in CatBoost alone, but the blend narrows the training-to-holdout gap. A single split cannot prove how the model will perform on every future dataset.

12. Error Analysis

The clean local holdout confusion matrix is shown below. Rows represent actual labels and columns represent predicted labels.

Actual \ Predicted	Sedan (0)	SUV (1)	Sports Car (2)
Sedan (0)	422	47	39
SUV (1)	48	448	2
Sports Car (2)	33	2	459

Sports Cars were the strongest class on this holdout, while Sedans were the most difficult. Many mistakes were between Sedan and SUV, which is reasonable because some numerical vehicle specifications can overlap between these categories.

13. Key Insights and Discussion

- Logistic Regression was a strong baseline even before blending.
- CatBoost added nonlinear tree-based predictions that were useful when combined with Logistic Regression.
- The depth-7 CatBoost configuration with 650 iterations, learning rate 0.035, L2 leaf regularization 5.0, and 2.5% training-noise augmentation was the best submitted setup.
- Not every added model or technique helped. SVM blending, test-time augmentation, and stacking gave lower Kaggle scores in the tested versions.
- The final model is a simple weighted probability blend, so it is easier to explain than a stacking pipeline.
- The 0.90600 Kaggle score is the final observed competition result; local validation scores remain separate diagnostics.

14. Limitations

- The training data is synthetic and may not represent real-world vehicles.
- The hidden evaluation data includes measurement uncertainty. Our added noise is a simple approximation, not a measured noise model.
- The local holdout accuracy (88.60%) is lower than the Kaggle score (90.60%); the evaluation sets are different, so the figures are not directly interchangeable.
- Repeatedly trying candidates against a public leaderboard can overfit decisions to that leaderboard.
- The model is predictive, not causal; feature importance or deeper interpretability analysis was not part of the final pipeline.

15. Future Work

- Use repeated duplicate-aware cross-validation to estimate score stability more reliably.
- Run a one-factor-at-a-time ablation study for CatBoost depth, learning rate, regularization, and noise strength.
- Use SHAP or feature importance to explain which vehicle measurements contribute most to each class prediction.
- Estimate the measurement-noise distribution more carefully before changing the augmentation strength.
- Tune any future blend weights using out-of-fold probabilities only, rather than repeated leaderboard guessing.

16. Conclusion

This project built an end-to-end multiclass vehicle classification pipeline, from data inspection and duplicate handling to model comparison and Kaggle submission. The final solution combines degree-2 Logistic Regression with a tuned CatBoost model through a 70:30 probability blend.

The selected CatBoost configuration used depth 7, 650 iterations, learning rate 0.035, L2 leaf regularization 5.0, random seed 42, and 2.5% training-noise augmentation. The final submitted prediction file achieved a Kaggle leaderboard score of 0.90600 (90.6%), exceeding the 90.5% target.